Advanced Programming Techniques Lab

Team Information

Members:

- Marios Ioannis Papadopoulos 1092834
- Filippos Neofytos Theologos 1092633
- Xristina Tzouda 1097346

SECTION A - RUNBOOK

Nesessary hardware and software from previous labs

- Hardware:
  - Raspberry Pi 5
  - HC-SR501 PIR motion sensor
  - Jumper wires(female to female)
- Wiring the sensor: Use the example given on lab02, made sure to connect the OUT on the same pin.
- Connection Due to bad connection, we weren't able to download `homeassistant` during lab time and by using ssh, so we worked on the raspberry.
- Software:
  - The PIR sensor logic (`sampler.py`, `interpreter.py`) is reused from Lab 02 and placed inside `pirlib/`.
  - Use a venv just like lab01 and lab02.
  - Make sure to inastall a `requirments.txt`.
  - Install Mosquitto brocker. Instructions givel on lab06

Part 1 — Run Home Assistant in Docker

1. Create a directory for Home Assistant configuration:

```
mkdir -p ~/homeassistant/config
```

2. Run home assistant:

```
docker run -d \
  --name homeassistant \
  --restart unless-stopped \
  -v ~/homeassistant/config:/config \
  -v /run/dbus:/run/dbus:ro \
  --network host \
  ghcr.io/home-assistant/home-assistant:stable
```

3. When it starts run :

```
docker logs -f homeassistant
```

4. We run it directly on the raspberry so we used the following line on the browser `http://localhost:8123`.

Part 2 — Connect Home Assistant to your MQTT broker

- Create the Mosquitto configuration file:

```
sudo nano /etc/mosquitto/conf.d/default.conf
```

- Add:

```
listener 1883
allow_anonymous true
```

- Restart Mosquitto:

```
sudo systemctl restart mosquitto
```

- Then we followed the instructions given on the lab course in order to add the MQTT on home assistant.

Verifying the connection

- Follow this path on the home assistant :
  - Settings → Devices & Services → MQTT
- Subscribe to # (meaning all topics)
- Publish the:
  - Topic : `test/ha`
  - Payload: **hello from Home Assistant**
- The payload should be visible

- Publish from the terminal:
  - `mosquitto_pub -h localhost -t "test/from-terminal" -m "hello from terminal"`
- It showed up in the Home Assistant MQTT listener, so the bridge is working.

Part 3 — Understand MQTT Discovery

Create motion sensor entity:

1. Publish a configuration message to `homeassistant/binary_sensor/pir_01_motion/config`
2. Write the following payload:

```
{
  "name": "PIR Motion Sensor",
  "state_topic": "smartbin/bin-01/pir-01/motion",
  "payload_on": "detected",
  "payload_off": "clear",
  "device_class": "motion",
  "unique_id": "pir_01_motion",
  "device": {
    "identifiers": ["pir-01"],
    "name": "PIR Sensor 01",
    "model": "HC-SR501",
    "manufacturer": "Generic"
  }
}
```

This tells the home assistant to create a binary sensor, something that is On or OFF. It also says that the if the topic receives "detected", the entity is ON and if it receives "clear", it is OFF. Try it from terminal:

```
mosquitto_pub -h localhost -t "homeassistant/binary_sensor/pir_01_motion/config" -
r -m '{
  "name": "PIR Motion Sensor",
  "state_topic": "smartbin/bin-01/pir-01/motion",
  "payload_on": "detected",
  "payload_off": "clear",
  "device_class": "motion",
  "unique_id": "pir_01_motion",
  "device": {
    "identifiers": ["pir-01"],
    "name": "PIR Sensor 01",
    "model": "HC-SR501",
    "manufacturer": "Generic"
  }
}'
```

- Publish a state update:

```
mosquitto_pub -h localhost -t "smartbin/bin-01/pir-01/motion" -m "detected"
```

A new entity is visible **PIR motion sensor** showing **detected**.

- Publish a second state update but instead of `detected` write `clear`. The same result shoud be shown.

Part 4 — Create the Smart Wastebin entity

- We created the `PIR motion sensor` in part 3.
- Create the wastebin by typing this:

```
mosquitto_pub -h localhost -t "homeassistant/sensor/wastebin_01_status/config" -r
-m '{
  "name": "Wastebin Status",
  "state_topic": "smartbin/bin-01/status",
  "value_template": "{{ value_json.state }}",
  "json_attributes_topic": "smartbin/bin-01/status",
  "unique_id": "wastebin_01_status",
  "device": {
    "identifiers": ["bin-01"],
    "name": "Smart Wastebin 01",
    "model": "Smart Wastebin v1",
    "manufacturer": "ECE CK801 Team"
  }
}'
```

Then publish state with attributes:

```
mosquitto_pub -h localhost -t "smartbin/bin-01/status" -m '{
  "state": "active",
  "location": "Lab Room 101",
  "last_motion": "2026-04-10T14:32:01Z",
  "total_events_today": 42
}'
```

- Create a motion event counter :

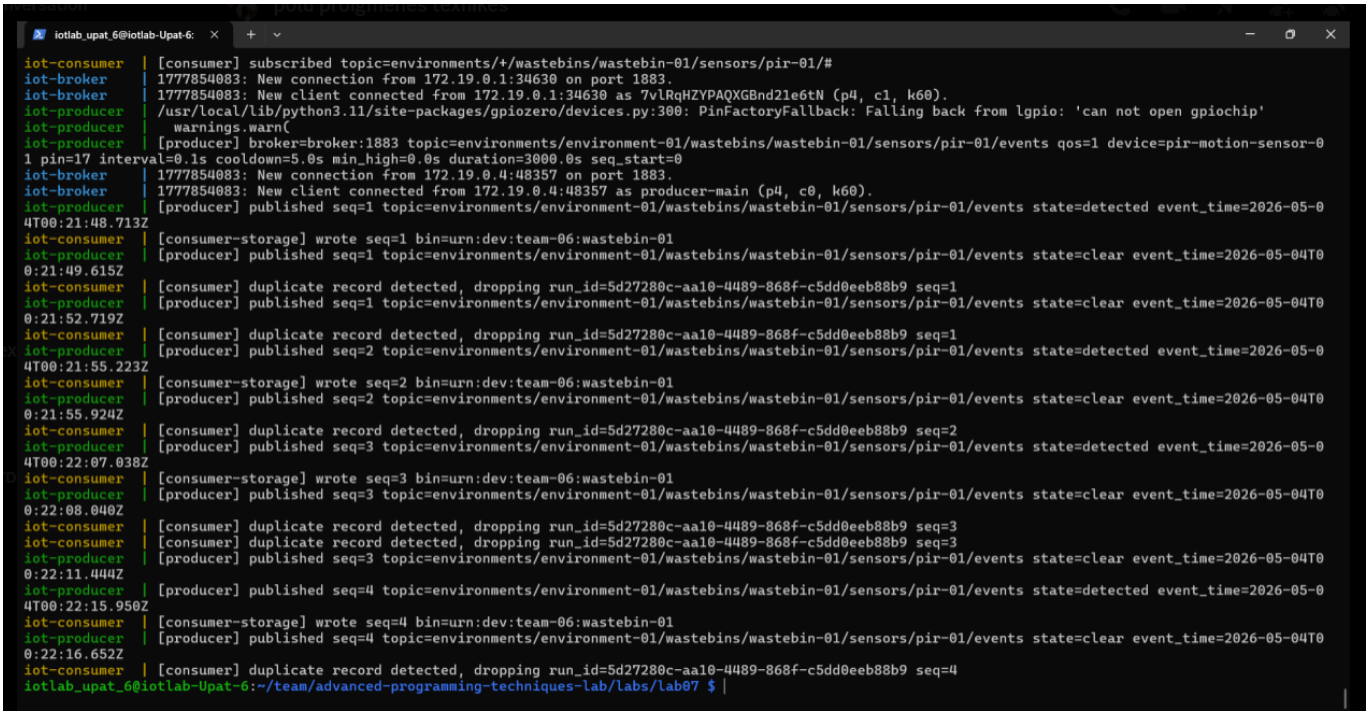
```
mosquitto_pub -h localhost -t
"homeassistant/sensor/wastebin_01_motion_count/config" -r -m '{
  "name": "Motion Event Count",
  "state_topic": "smartbin/bin-01/pir-01/event_count",
  "unit_of_measurement": "events",
  "icon": "mdi:motion-sensor",
  "unique_id": "wastebin_01_motion_count",
  "device": {
    "identifiers": ["bin-01"],
```

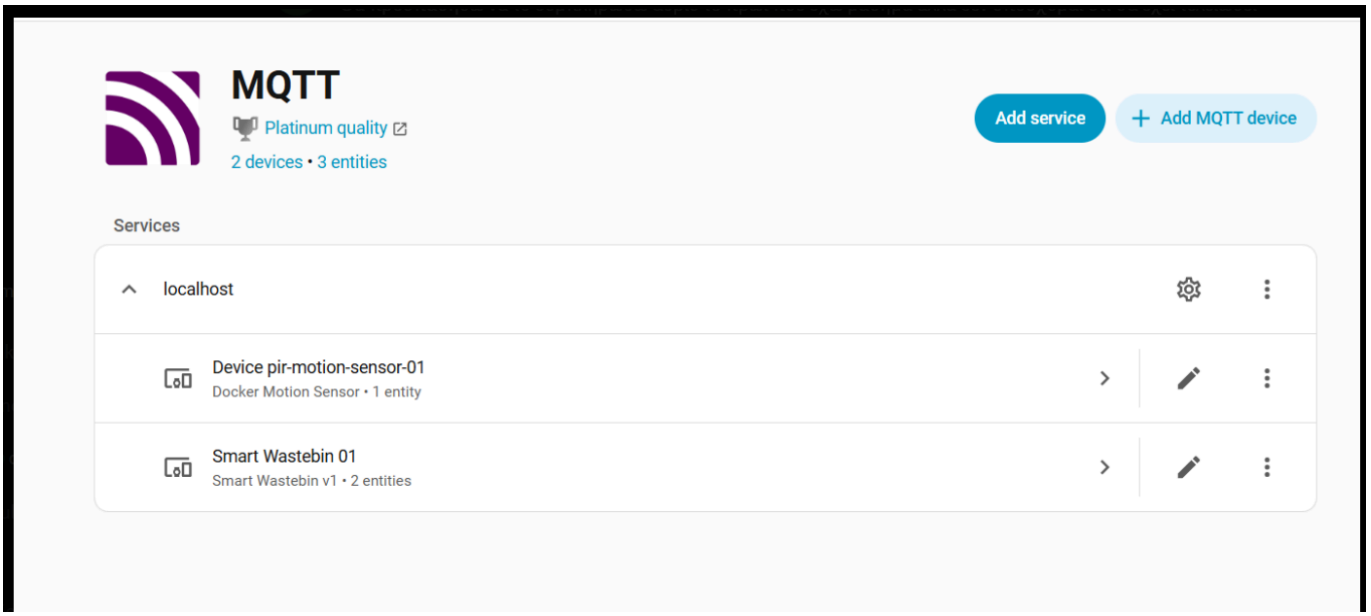
```
      "name": "Smart Wastebin 01"
    }
  }'
```

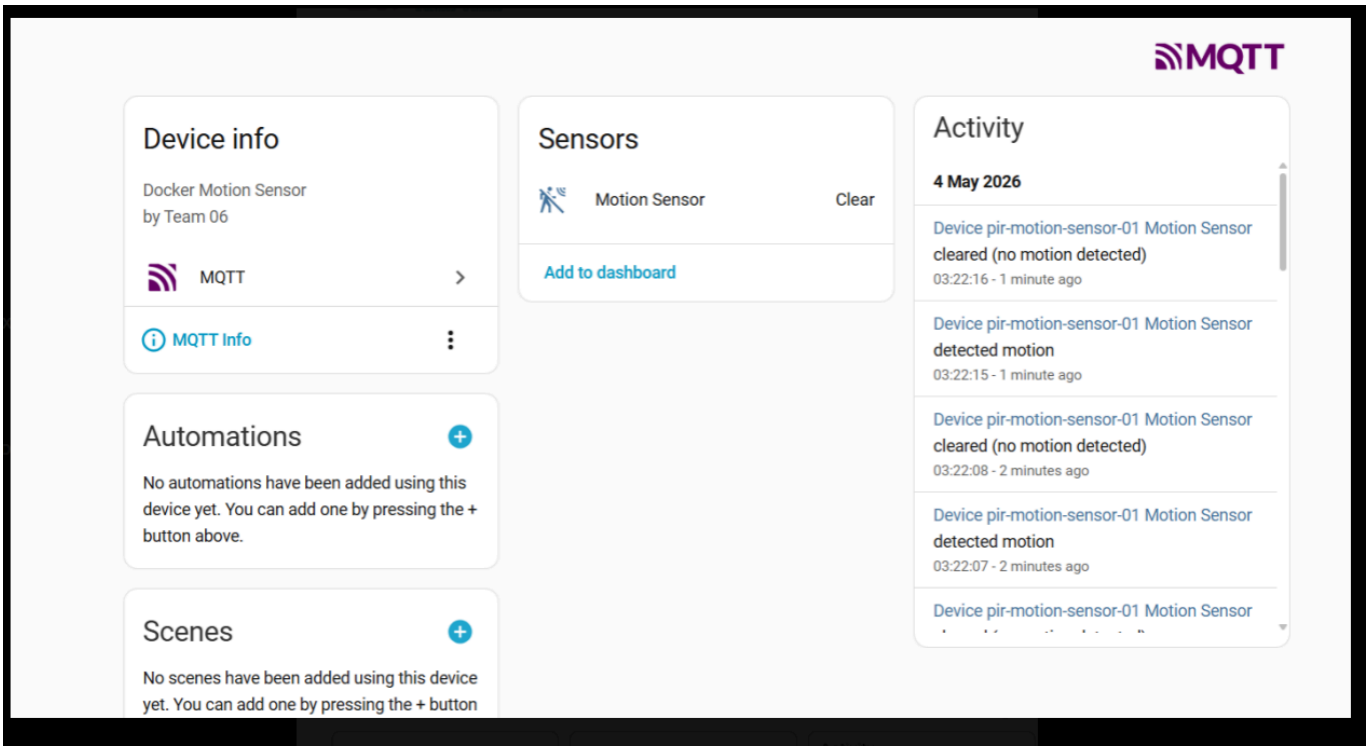
- We also created a helper counter, that runs in the home assistant called **Wastebin Motion Count**

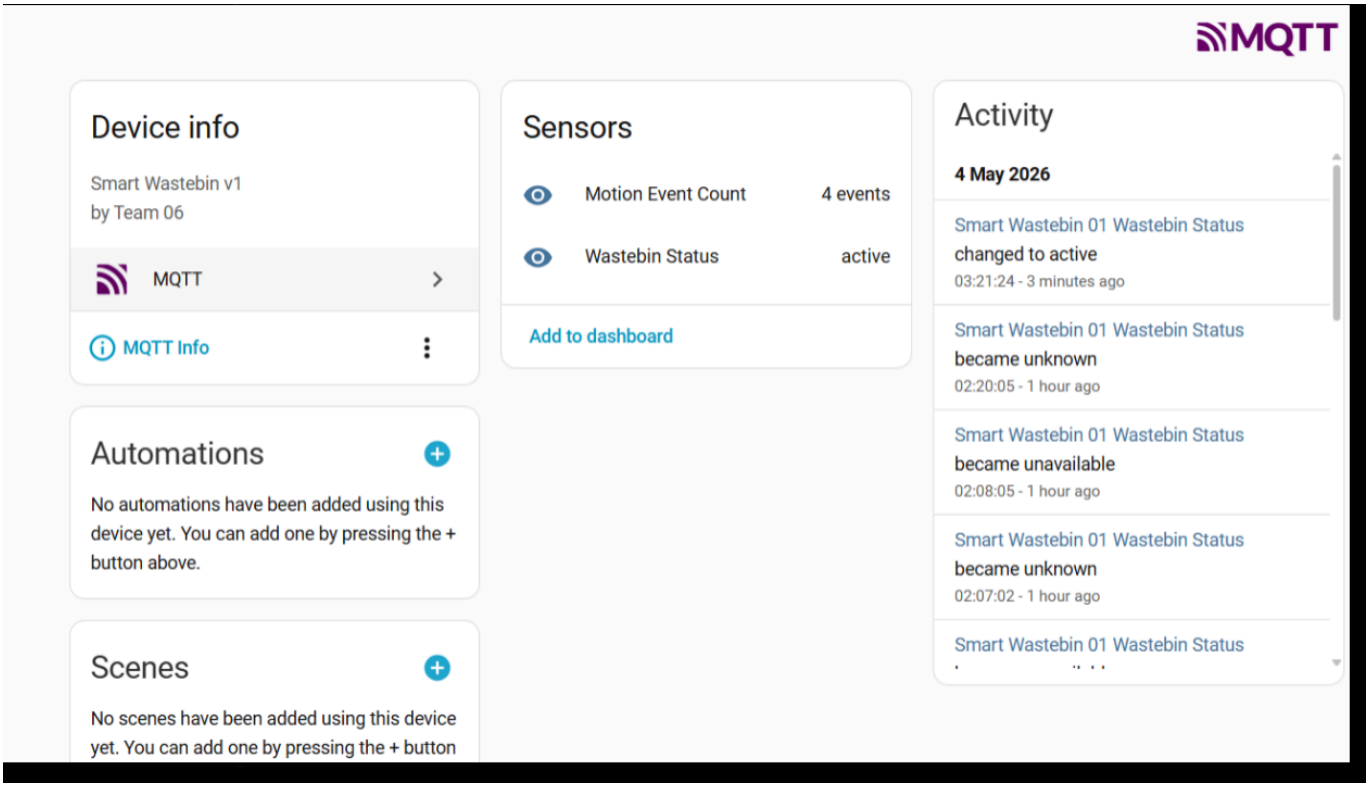
Part 5 — Publish from your pipeline

We changed the code on the `producer.py` and got the following results:





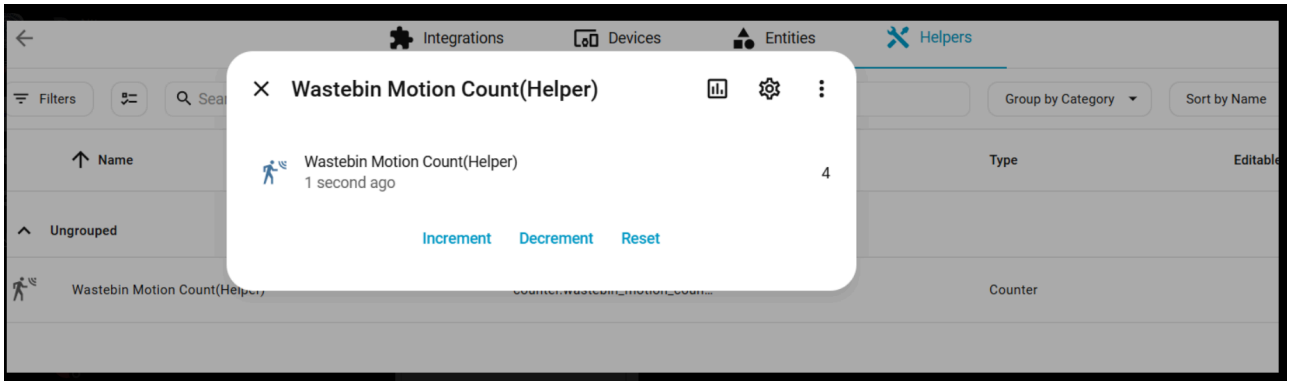


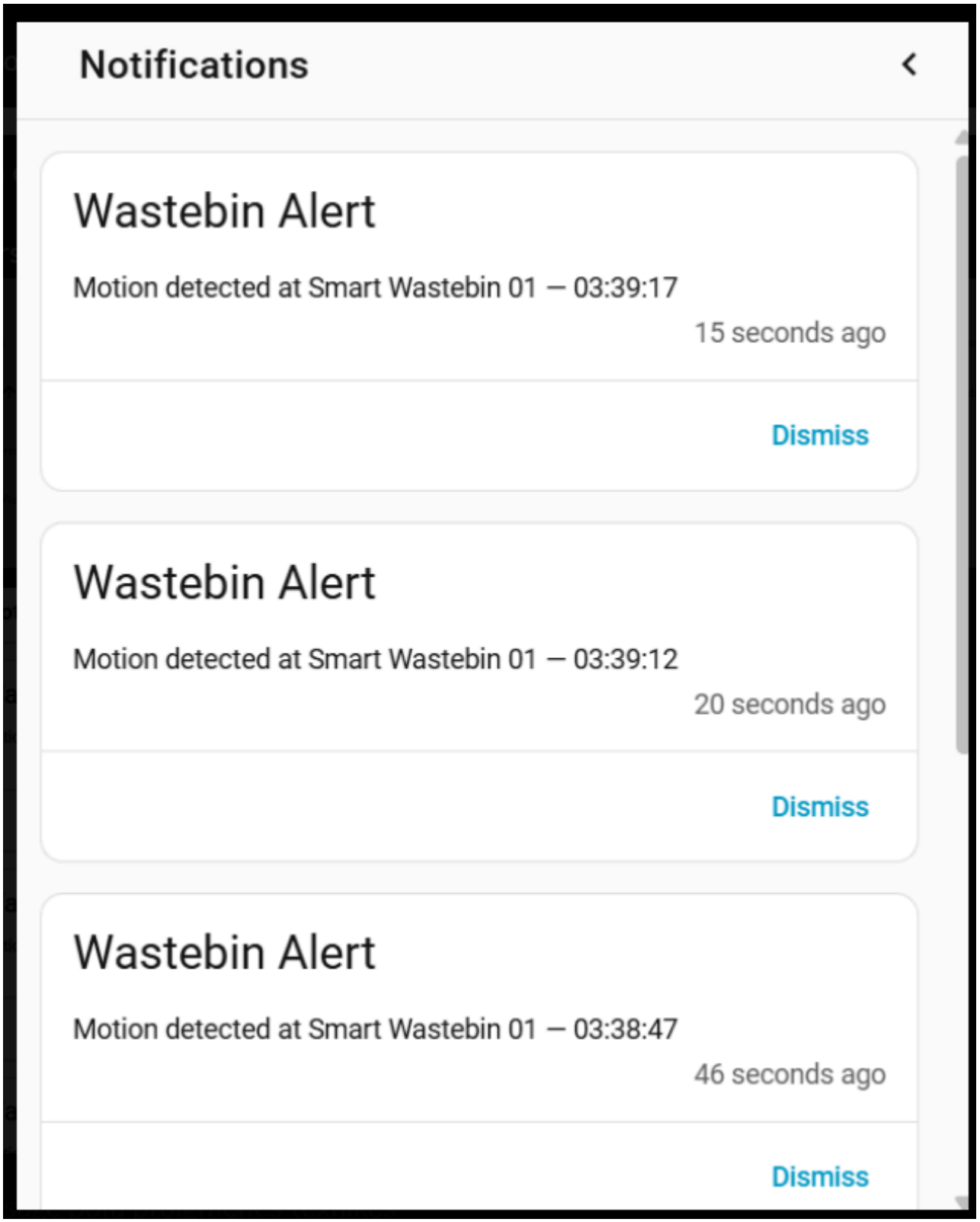


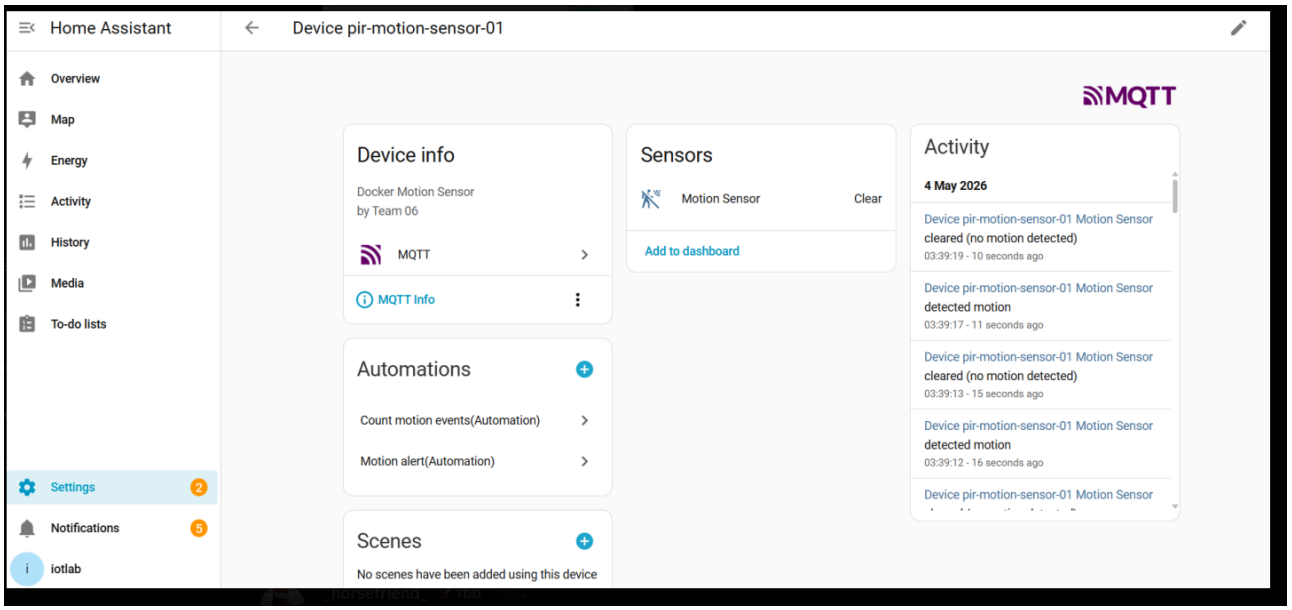
Part 6 — Create a motion counter with automations

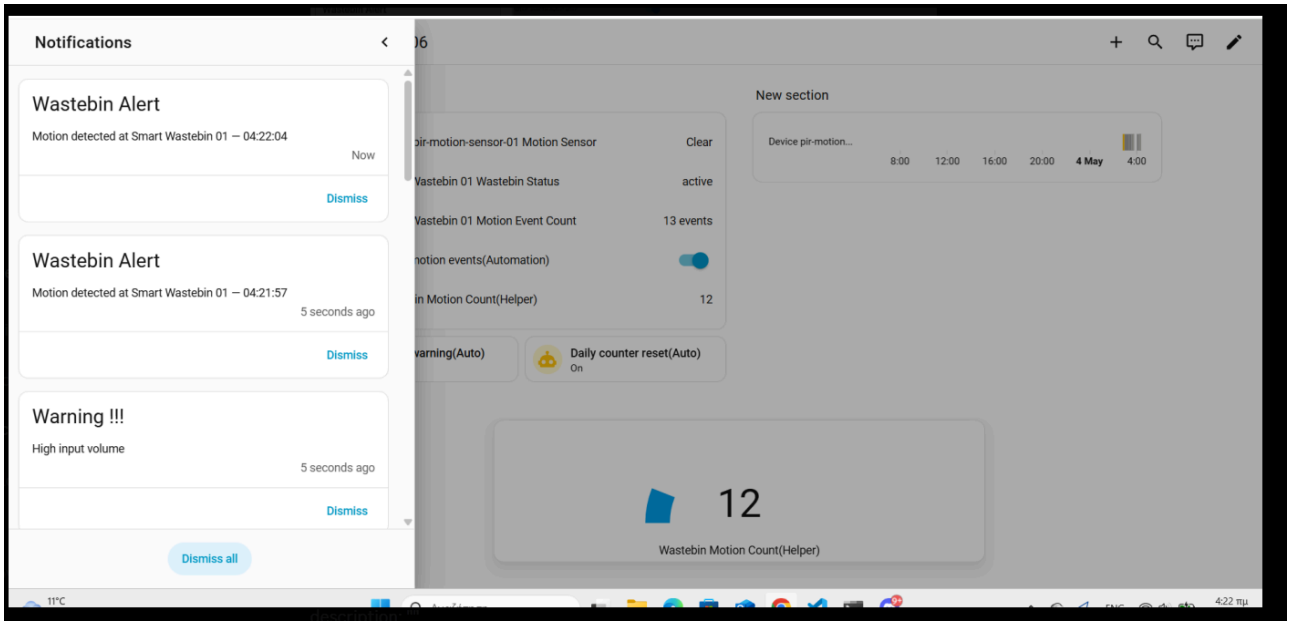
- Following the steps given on the lab website we created the helper counter running on the home assistant app named **Wastebin Motion Count**

- The results:


Integrations
Devices
Entities
Helpers
Filters
Group by Category
Sort by Name
Wastebin Motion Count(Helper)
Wastebin Motion Count(Helper)
1 second ago
4
Increment
Decrement
Reset
Name
Type
Ungrouped
Counter


Notifications
Wastebin Alert
Motion detected at Smart Wastebin 01 — 03:39:17
15 seconds ago
Dismiss
Wastebin Alert
Motion detected at Smart Wastebin 01 — 03:39:12
20 seconds ago
Dismiss
Wastebin Alert
Motion detected at Smart Wastebin 01 — 03:38:47
46 seconds ago
Dismiss





The YAML we created

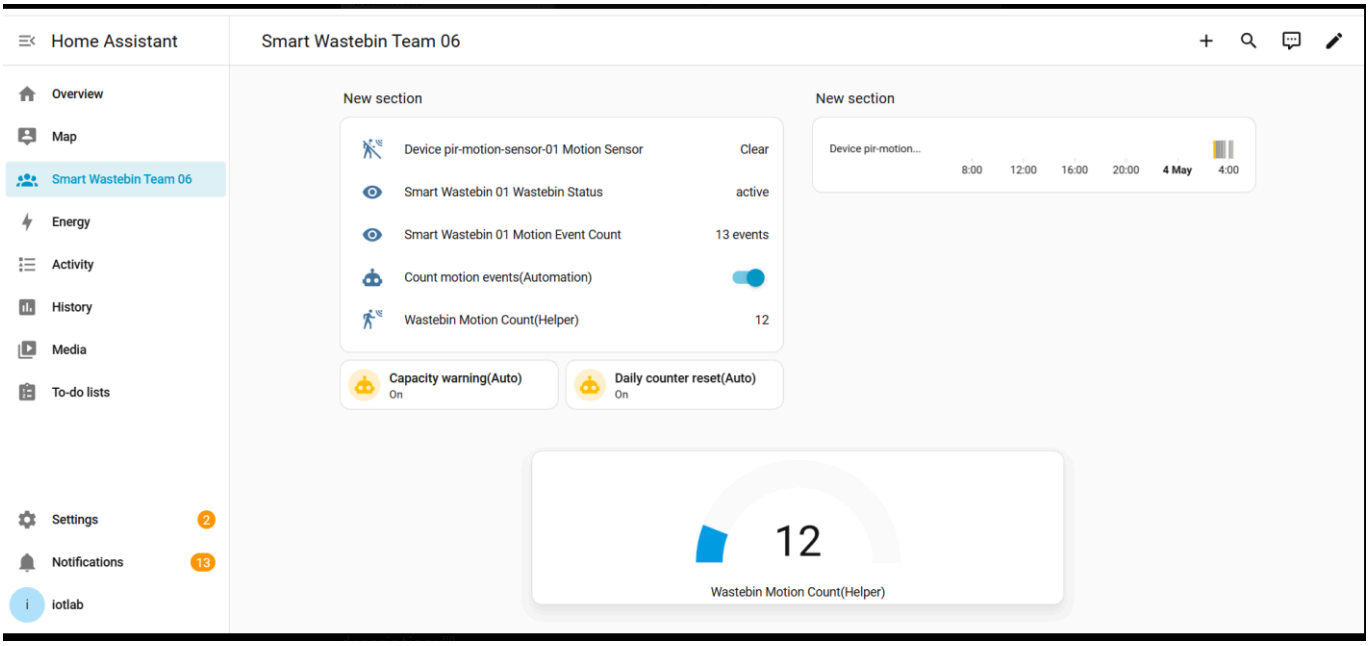
```
alias: Motion alert(Automation)
description: ""
triggers:

trigger: state
  entity_id:
binary_sensor.device_pir_motion_sensor_01_motion_sensor
to:
"on"
conditions: []
actions:

action: persistent_notification.create
  metadata: {}
  data:
    message: Motion detected at Smart Wastebin 01 — {{ now().strftime('%H:%M:%S')
}}
```

```
    title: Wastebin Alert
mode: single
-------------------------------
alias: Daily counter reset(Auto)
description: ""
triggers:

trigger: time_pattern
  hours: "0"
  minutes: "00"
  seconds: "00"
conditions: []
actions:

action: counter.reset
  metadata: {}
  target:
    entity_id: counter.wastebin_motion_count_helper
  data: {}
mode: single
--------------------------------
alias: Count motion events(Automation)
description: ""
triggers:

trigger: state
  entity_id:
binary_sensor.device_pir_motion_sensor_01_motion_sensor
to:
"on"
conditions: []
actions:

action: counter.increment
  metadata: {}
  target:
    entity_id: counter.wastebin_motion_count_helper
  data: {}
mode: single
-------------
alias: Capacity warning(Auto)
description: ""
triggers:

trigger: numeric_state
  entity_id:
counter.wastebin_motion_count_helper
above: 10
conditions: []
actions:

action: persistent_notification.create
  metadata: {}
  data:
```

```
      title: Warning !!!
      message: High input volume
  mode: single
```

Part 7 — Build a simple dashboard

Our dashboard:



SECTION B - REPORT

RQ1

Home Assistant is an open-source local IoT platform with a web dashboard, device registry, automation engine, and state history built in. This practically means we only needed to connect our pipeline to it via MQTT.

RQ2

Home Assistant OS takes over the entire machine with its own operating system. Home Assistant Container runs only the HA core as a Docker container on our existing OS.That's why we used the container method

RQ3

An entity is anything with a state, like a sensor reading, a device status or a counter. 3 examples from our setup:

1. Binary sensor : `PIR_Motion_Sensor` -> state: clear/detected.
2. Sensor : `Wastebin_status` -> state: active.
3. Counter : `Wastebin_Motion_Count` -> state: Number (how many rubbish has been thrown).

RQ4

Our system publishes a retained MQTT discovery message to `homeassistant/<component>/<object_id>/config`. The payload defines the entity name, state topic, payload values, device class, and device metadata that Home Assistant uses to create the entity automatically.

RQ5

The retain flag makes the broker store the last message. If HA restarts after the discovery message was published, it reads the retained message on reconnect and recreates all entities. Without retain, entities would disappear every time HA restarts.

RQ6

The device block groups entities under a physical device using shared identifiers. When multiple entities share the same device.identifiers, HA groups them all under one device entry(e.g smart_wastebin_01

RQ7

`state_topic` provides the entity's primary state value. `json_attributes_topic` provides supplementary key-value data attached to that state. Use state_topic for the main value and `json_attributes_topic` for extra context like location or last_motion.

RQ8

Entity	Type	State Topic	Reason
PIR Motion Sensor	`binary_sensor`	`smartbin/bin-01/pir-01/motion`	Represents the physical PIR sensor; binary state: detected/clear
Wastebin Status	`sensor`	`smartbin/bin-01/status`	Holds the overall bin state
Motion Event Count	`sensor`	`smartbin/bin-01/pir-01/event_count`	Tracks cumulative motion events published by the producer
Wastebin Motion Count (Helper)	`counter`	*(HA helper, no MQTT topic)*	HA-native counter incremented by automation and shown on the dashboard
Automations are not listed here because they are workflows, not entities, and they do not have MQTT state topics.			

RQ9

`device_class: motion` changes the icon to a motion sensor, changes the state labels from "On/Off" to "Detected/Clear", and helps Home Assistant categorize the entity in dashboards and history.

RQ10

- **Motion Event Count** — a sensor that tracks cumulative motion events published directly from the producer. This gives a running total that persists independently of the HA counter helper.
- **Wastebin Motion Count (Helper)** — a HA-native counter incremented by automation. This can be reset daily by the Daily counter reset automation, giving a per-day usage count, and it displays nicely as a gauge on the dashboard.

RQ11

Group	Entities
Device1: Smart Wastebin	Motion Event Count Wastebin Status
Device2: Docker Motion Sensor	Motion Sensor
Standalone entities	wastebin motion count (helper)

RQ12

The Counter helper is a built-in HA entity that holds a persistent integer value. It survives restarts when "Restore" is enabled. The following services can be called on it:

- `counter.increment` — adds the step value (default 1)
- `counter.decrement` — subtracts the step value
- `counter.reset` — sets the value back to the initial value (0)
- `counter.set_value` — sets it to a specific integer

RQ13

```
alias: Count motion events(Automation)
description: ""
triggers:

trigger: state
  entity_id:
binary_sensor.device_pir_motion_sensor_01_motion_sensor
    to: "on"
conditions: []
actions:

action: counter.increment
  metadata: {}
  target:
    entity_id: counter.wastebin_motion_count_helper
  data: {}
mode: single
```

- Trigger: PIR sensor state changes to "on" (motion detected)
- Conditions: none — every detection counts
- Action: increments counter.wastebin_motion_count_helper by 1

RQ14

Three additional automations were created:

1. **Motion Alert** — Trigger: PIR sensor goes to `"on"`. It creates a persistent notification with the current timestamp.

```
alias: Motion alert(Automation)
description: ""
triggers:

trigger: state
  entity_id:
binary_sensor.device_pir_motion_sensor_01_motion_sensor
to:
"on"
conditions: []
actions:

action: persistent_notification.create
  metadata: {}
  data:
    message: Motion detected at Smart Wastebin 01 – {{ now().strftime('%H:%M:%S')
}}
    title: Wastebin Alert
mode: single
```

2. **Daily Counter Reset** — Trigger: `time_pattern` at midnight. It resets `wastebin_motion_count_helper` to 0 every day.

```
alias: Daily counter reset(Auto)
description: ""
triggers:

trigger: time_pattern
  hours: "0"
  minutes: "00"
  seconds: "00"
conditions: []
actions:

action: counter.reset
  metadata: {}
  target:
    entity_id: counter.wastebin_motion_count_helper
  data: {}
mode: single
```

3. **Capacity Warning** — Trigger: counter exceeds 10. It creates a persistent notification warning of high input volume.

```
alias: Capacity warning(Auto)
description: ""
triggers:

trigger: numeric_state
  entity_id:
counter.wastebin_motion_count_helper
above: 10
conditions: []
actions:

action: persistent_notification.create
  metadata: {}
  data:
    title: Warning !!!
    message: High input volume
mode: single
```

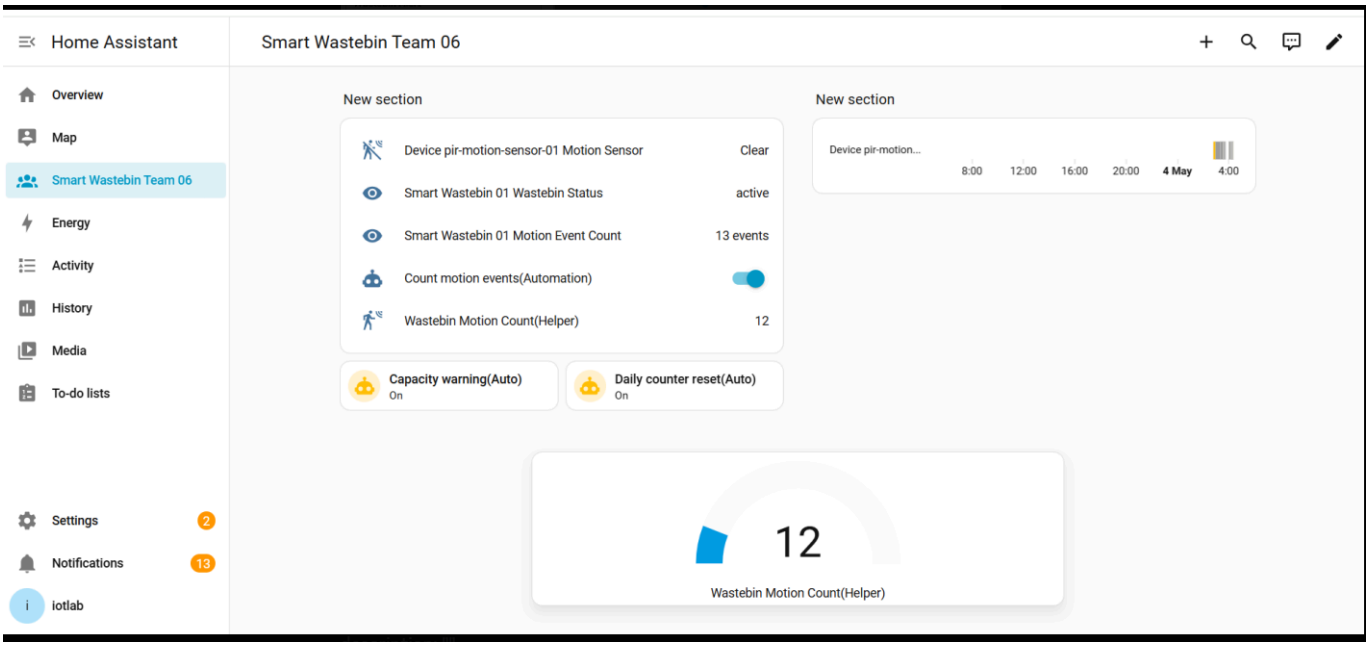
RQ15

The Capacity Warning automation is exactly this.It is treggered when the counter value rises above 10 and sends a notification that the bin has high input volume.

RQ16

The JSONL consumer needs the full structured event (timestamps, sequence number, latency, JSON-LD context). HA needs only a simple scalar like "detected". Sending the full JSON event to HA's state topic would require complex value_template workarounds and would tightly couple HA's config to the event schema. Separate topics let each consumer get exactly what it needs.

RQ17



RQ18

The entity stays at its last known state and it does not become unavailable automatically. In our setup, the discovery config already includes an `availability_topic` with `online`/`offline` payloads. To make the entity switch to unavailable when the producer goes down, the producer must publish `offline` on shutdown or use an MQTT Last Will and Testament (LWT) so the broker sends `offline` on an unexpected disconnect. Home Assistant will then correctly show the entity as unavailable.

RQ19

Home Assistant saves weeks of development because you get a dashboard, state history, automations, and alerting for free by just downloading the app. The trade-off is working within its conventions (discovery topics, device classes, value templates) with limited control over the underlying framework.

RQ20

Running locally means automations and state updates execute in milliseconds with no internet dependency. The system keeps working during outages, sensor data never leaves the premises and there are no ongoing cloud costs.

RQ21

With MQTT discovery, each new bin just publishes its own config messages at startup and HA auto-registers all 30 entities with no changes to HA's configuration. With manual YAML, every sensor requires editing config files and restarting HA.